

BCA 5th Semester

PHP

Topic: Forms Handling and User Input

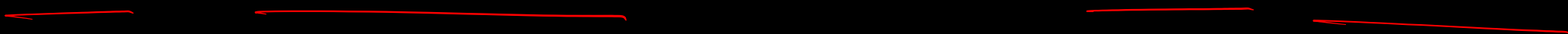
- By Soudip Sir

3

Forms Handling and User Input: Handling form data, GET vs. POST methods, validating and sanitizing user input

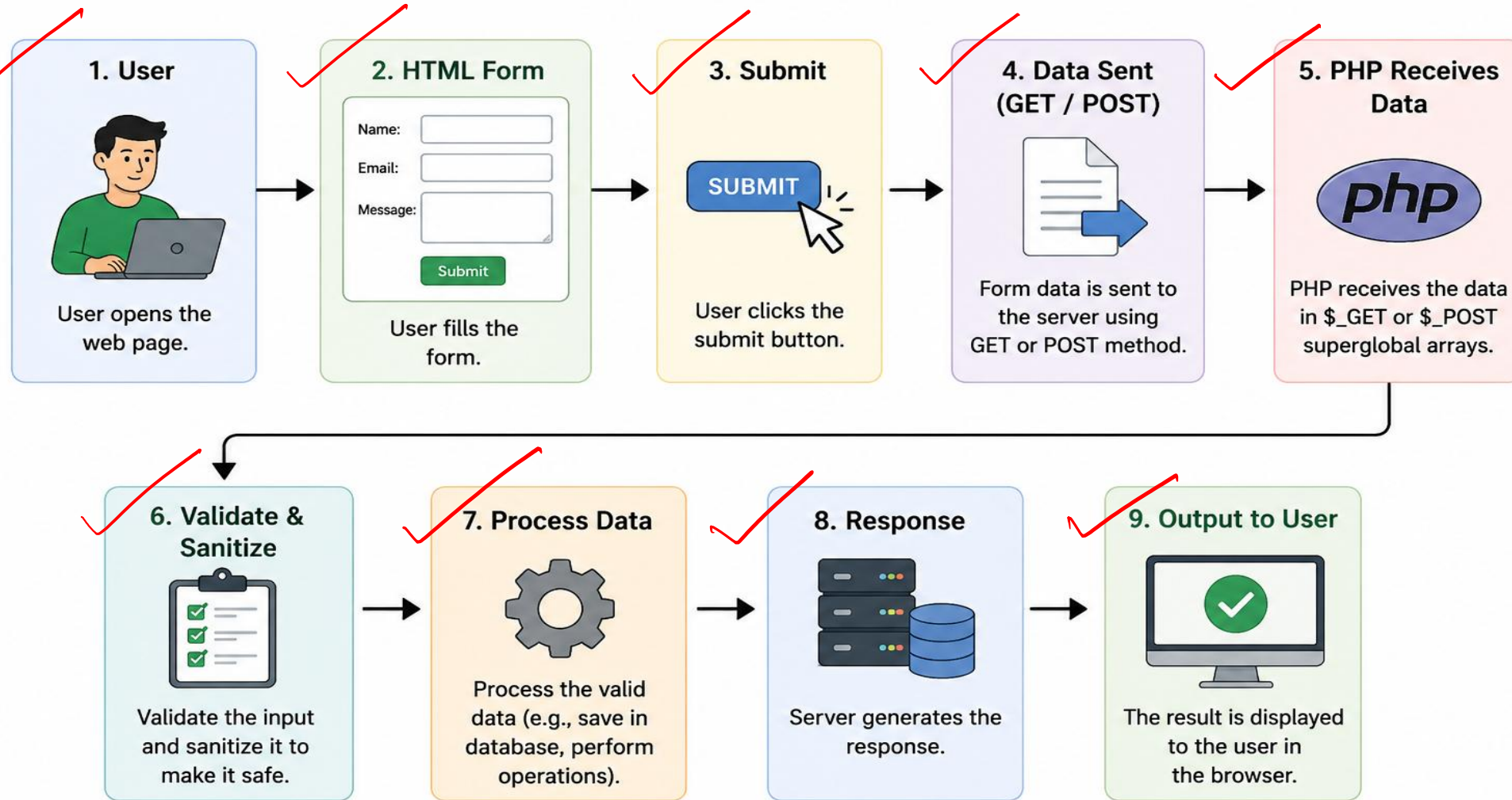
PHP Form Handling

Form handling is the process of collecting and processing information that users submit through HTML forms. In PHP, we use special tools called `$_POST` and `$_GET` to gather the data from the form. Which tool to use depends on how the form sends the data—either through the POST method (more secure, hidden in the background) or the GET method (data is visible in the URL).

The text at the bottom of the paragraph is underlined with red lines. The underlines are not perfectly straight, with some segments being longer than others, and they extend across the width of the text.

- ✓ **Collecting Data:** Retrieving form data using PHP.
- ✓ **Validating Data:** Ensuring that the input meets expected formats.
- ✓ **Sanitizing Data:** Cleaning up the data to prevent malicious content.
- ✓ **Processing Data:** Using the data for its intended purpose
(e.g., saving to a database, sending an email, etc.).
- ✓ **Returning a Response:** Displaying feedback to the user or redirecting them to another page.

PHP Forms Handling Process



Form Attributes

- ✓ **action:** The action attribute specifies the URL where the form data will be sent when the form is submitted.

```
<form method="post" action="process_form.php">
```

- ✓ **method:** The method attribute specifies the HTTP method (GET or POST) to use when sending form data.

```
<form method="post" action="process_form.php">
```

- ✓ **name:** The name attribute is crucial in PHP form handling, as it is used to refer to the data submitted by the form fields.

```
$username = $_POST['username']; // Accessing the form data
```

- ✓ **target:** The target attribute specifies where to display the response after submitting the form. It determines where the resulting page (or response) will appear once the form is submitted.

```
<form method="post" action="process_form.php" target="_blank">  
  <input type="text" name="username" required>  
  <input type="submit" value="Submit">  
</form>
```

- ✓ **enctype:** The enctype (encoding type) attribute defines how the form data should be encoded when submitted to the server. This is particularly important when submitting forms that include file uploads. The enctype attribute specifies how form data should be encoded before sending it to the server. **It only works when the form's submission method is set to method="POST".**

Form Elements

Form processing contains a set of controls through which the client and server can communicate and share information. The controls used in forms are:

- ✓ **Input Field:** Input field is the most common form element, allowing users to input a single line of text, such as their name, address, or any other simple text information.

```
<input type="text" name="fullname" required>
```

- ✓ **Password Input Field:** The password input field hides the text entered, making it suitable for secure data entry like passwords.

```
<input type="password" name="password" required>
```

- ✓ **Checkboxes:** Checkboxes allow users to select multiple options from a set of choices. They are often used for lists of features or permissions.

```
<input type="checkbox" name="subscribe" value="yes"> Subscribe to newsletter
```

- ✓ **Radio Buttons:** Radio buttons allow the user to choose only one option from a set of predefined options. This is useful for binary choices, such as gender selection.

`<input type="radio" name="gender" value="female"> Female`

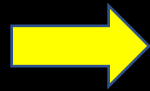
`<input type="radio" name="gender" value="male"> Male`

- ✓ **Textarea:** The textarea element allows users to input multiple lines of text, making it useful for longer messages, feedback, or comments.

`<textarea name="message" rows="5" cols="40" required></textarea>`

Feature	GET ✓	POST ✓
Data location	URL/query string ✓	HTTP request body ✓
Visibility	Data appears in URL ✓	Data isn't placed in URL ✓
Data size	Limited by URL/request limits ✓	Generally suitable for larger payloads ✓
Bookmarking	Request URL can be bookmarked with parameters ✓	Not normally bookmarkable in the same way ✓
Browser history	Parameters may appear in history ✓	Data isn't normally stored in URL history ✓
Caching	GET requests may be cached ✓	POST requests are generally not cached by default ✓
Typical use	Search, filtering, retrieving data ✓	Login, registration, creating/updating data ✓
PHP superglobal	\$_GET ✓	\$_POST ✓
Security	Not inherently secure <i>Low</i>	Not inherently secure <i>Most</i>
HTTPS required for confidentiality?	Yes	Yes

Handling Form Data



1. Create HTML form ✓✓



2. User enters data ✓✓



3. Submit form ✓✓



4. PHP receives data ✓✓



5. Check whether data exists ✓✓



6. Validate data ✓✓



7. Sanitize data when appropriate ✓✓



8. Process/store data ✓✓



9. Send response ✓✓

Checking Whether Data Was Submitted

Before accessing form data, it is good practice to check whether the expected key exists.

For example:

```
if (isset($_POST["name"])) {
```

```
    $name = $_POST["name"];
```

```
    echo $name;
```

```
}
```

isset() checks whether a variable/key exists and is not null.

Checking the Request Method

PHP provides:

`$_SERVER["REQUEST_METHOD"]` ✓✓

Example:

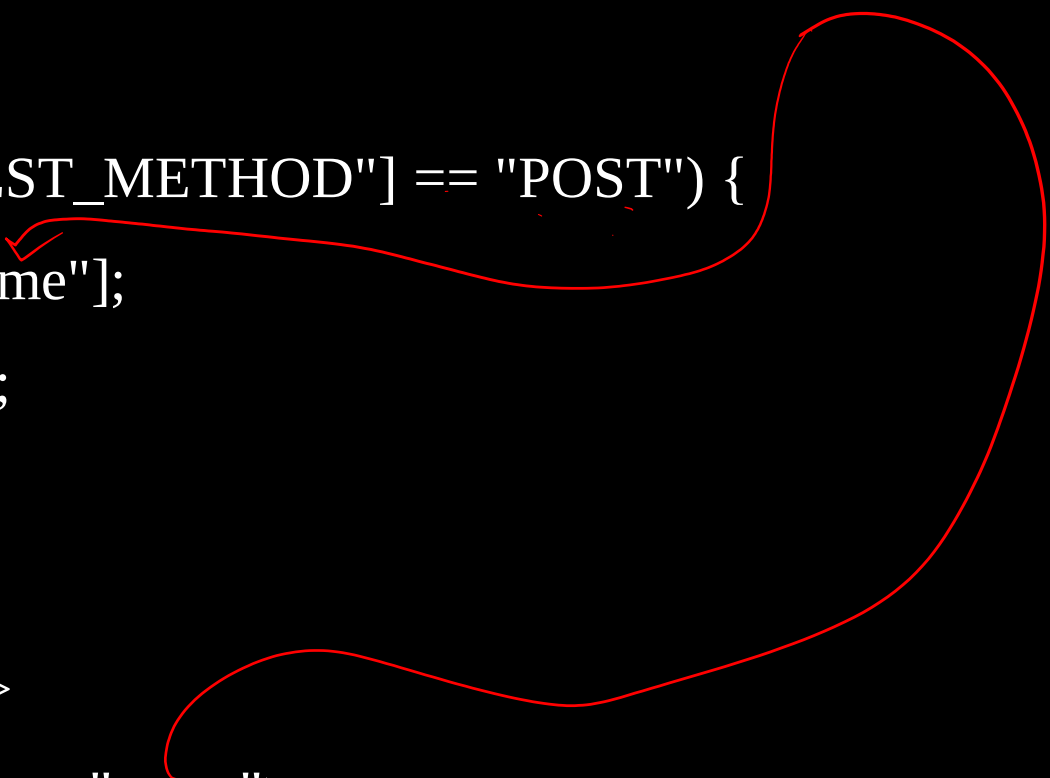
```
if ($_SERVER["REQUEST_METHOD"] == "POST") {  
    echo "Form submitted using POST";  
}
```

This is useful when the same PHP file displays the form and processes it.

Example:

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST["name"];
    echo "Hello " . $name;
}
?>

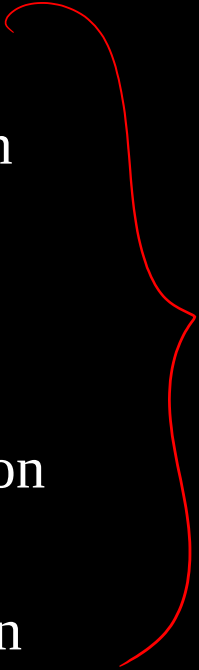
<form method="POST">
    <input type="text" name="name">
    <button type="submit">Submit</button>
</form>
```



PHP Validation

Form validation is a crucial step that needs to be completed before submitting data to the database. This process is performed to prevent any errors or faulty data from being inserted into the database. The HTML form includes various input fields such as email, text, checkbox, radio button, etc. These input fields must be validated to ensure that the user enters only the intended values and not incorrect ones.

Table of Content

- ✓ • Empty Field Validation
 - ✓ • String Validation
 - ✓ • Number Validation
 - ✓ • Email Validation
 - ✓ • String Length Validation
 - ✓ • URL Validation
 - ✓ • Button Click Validation
- 

Empty Field Validation

This is used to check if user has added any data in the required input field. Here, empty() is used to check if a variable is empty or not.

```
// Validating that the user name is added or not
if (empty($_POST["UserName"])) {
    $errorMsg = "User Name is required";
} else {
    // If username is added, we can use it for further use
    $userName = input_data($_POST["UserName"]);
}
```

String Validation

This is used to check if user has added valid string in the required input field. Here, preg_match() is used for pattern matching.

```
$userName = $_POST["userName"];  
// To check that User Name only contains alphabets, numbers, and underscores  
if (!preg_match("/^[a-zA-Z0-9_]*$/", $userName)) {  
    $errorMsg = "Only alphabets, numbers, and underscores are allowed for User Name";  
}else{  
    echo $userName;  
}
```

✓ Number Validation

This is used to check if user has added only numbers in the required input field.

```
$age = $_POST["age"];  
// To check that Phone No is well-formed  
if (!preg_match("/^[0-9]*$/", $age)) {  
    $errorMsg = "Only numeric values are allowed!!";  
}else{  
    echo $age;  
}
```

Email Validation

This is used to check if user has added a valid email in the required input field.

```
✓ $emailID = $_POST["emailID"];  
  
// This pattern is standard for email validation  
$pattern = "^[_a-z0-9-]+(\.[_a-z0-9-]+)*@[a-z0-9-]+(\.[a-z0-9-]+)*(\.[a-z]{2,3})$^";  
  
// To check that the e-mail address is well-formed  
if (!preg_match($pattern, $emailID) ){  
    $errorMsg = "Invalid Email ID format";  
}  
else{  
    echo $emailID;  
}
```

✓ String Length Validation

This is used to check if the length of the string is in valid range. Here, strlen() is used to find the length of the string.

```
$phoneNo = $_POST["phoneNo"];
```

```
// To check that length of Phone Number length should not be less and greater than 10
```

```
if (strlen($phoneNo) != 10) {
```

```
    $errorMsg = "Please provide a phone number of 10 digits!!";
```

```
}else{
```

```
    echo $phoneNo;
```

```
}
```

✓ URL Validation

This is used to check if user has added a valid URL in the required input field.

```
$url = $_POST["url"];  
// This pattern is standard for URL validation  
$pattern = "^b(?:(:https?|ftp):\\V|www\\.)[-a-z0-9+&@#\\/%?~_]|!:,.;]*[-a-z0-9+&@#\\/%=~_]/i"  
// To check that URL address syntax is valid  
if (!preg_match($pattern, $url)) {  
    $errorMsg = "You entered an INVALID URL";  
}else{  
    echo $url;  
}
```

Button Click Validation

This is used to check if user has pressed the required button. Here, isset() is used to check if a variable is declared and is not NULL.

```
if (isset($_POST['login'])) {  
    echo "LogIn button is clicked.";  
} else {  
    echo "LogIn button is not clicked.";
```

PHP Sanitizing

Sanitizing data means removing any illegal character from the data. Sanitizing user input is one of the most common tasks in a web application.

To make this task easier PHP provides native filter extension that you can use to sanitize the data such as e-mail addresses, URLs, IP addresses, etc.

✓ **The PHP Filter Extension:** PHP filters are used to sanitize and validate external input. The PHP filter extension has many of the functions needed for checking user input, and is designed to do data sanitization easier and quicker. This function, when using the flag in the example, is making sure that the code removes all characters except letters, digits and the following characters !#\$%&'*+,-=?_`{|}~@.[] .

Advantages of using Filters: Many web applications receive external input. External input/data can be:

- ✓ ● User input from a form
- ✓ ● Cookies
- ✓ ● Web services data
- ✓ ● Server Variables
- ✓ ● Database query results

Sanitizing a String: The following example uses the `filter_var()` function to remove all HTML tags from a string.

Program :

```
<?php
```

```
//Use of filter_var()
```

```
$str = "<h1>GeeksforGeeks</h1>";
```

```
$newstr = filter_var($str,  
    FILTER_SANITIZE_STRING);
```

```
echo $newstr;
```

```
?>
```

Output:

GeeksforGeeks

✓ **Sanitizing an Email Address:** The following example uses the `filter_var()` function to remove all illegal characters from the `$email` variable.

Program:

```
<?php
$email = "geeksforgeeks@gmail.co<m>";
// Remove all illegal characters
// from email
$email = filter_var($email,
    FILTER_SANITIZE_EMAIL);
echo $email;
?>
```

Output:

geeksforgeeks@gmail.com

✓ **Sanitizing a URL:** The following example uses the `filter_var()` function to remove all illegal characters from a URL.

Program:

```
<?php
$url = "www.geeksforgeeks.org";
// Remove all illegal characters
// from a url
$url = filter_var($url,
    FILTER_SANITIZE_URL);
echo $url;
?>
```

Output:

www.geeksforgeeks.org

Common Sanitization Techniques:

Function	Description
 htmlspecialchars()	Converts special characters to HTML entities
 strip_tags()	Removes HTML and PHP tags
 filter_var(\$data, FILTER_SANITIZE_STRING)	Removes unwanted characters
 trim()	Removes whitespace from beginning and end

Feature	PHP Validation	PHP Sanitization
Purpose	Ensures data is <u>correct</u>	Removes <u>harmful</u> or <u>unwanted</u> content
Action	Accept or reject input ✓	Clean/modify input ✓
Checks	<u>Format, type, range</u>	Unsafe <u>characters</u> , <u>HTML</u> tags
Security Role	Ensures <u>correctness</u>	Prevents attacks like <u>XSS</u> , <u>SQLi</u>
Example	VALIDATE_EMAIL	SANITIZE_STRING
Output	TRUE/FALSE (pass/fail)	Cleaned/filtered string
When Used	Before processing data ✓	Before storing or displaying data ✓✓

FORM HANDLING AND SUBMITTING PROCESS IN PHP

Form handling in PHP refers to the process of **collecting data from an HTML form, processing it using PHP, validating and sanitizing** the data, and then performing actions like **displaying, storing in database, sending email**, etc.

The form submission process occurs through **GET** or **POST** methods.

1. HTML Form Creation

HTML forms are used to collect data from users. The <form> tag contains:

- action → the PHP file that will receive the data
- method → GET or POST
- input fields → text, email, password, number, etc.

Example HTML Form

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<form action="process.php" method="POST">
```

```
  Name: <input type="text" name="name"><br><br>
```

```
  Email: <input type="email" name="email"><br><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

✓ 2. Submitting the Form

When the user clicks the **Submit** button:

- The browser collects all field values.
- Sends them to the **server**.
- The request is delivered to **process.php** (based on action="process.php").

3. Receiving Form Data in PHP

PHP receives form data through superglobal arrays:

- `$_GET` → if method="GET"
- `$_POST` → if method="POST"
- `$_REQUEST` → receives from both GET & POST

Example: Receiving Data

```
<?php
```

```
$name = $_POST['name'];
```

```
$email = $_POST['email'];
```

```
echo "Name: " . $name . "<br>";
```

```
echo "Email: " . $email;
```

```
?>
```

✓ 4. Validation and Sanitization

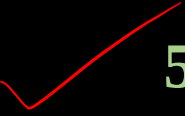
To ensure data integrity and security:

Sanitize

✓ `$name = filter_var($_POST['name'], FILTER_SANITIZE_STRING);`

Validate

```
if (!filter_var($_POST['email'], FILTER_VALIDATE_EMAIL)) {  
    echo "Invalid Email Format";  
}
```



5. Checking Form Submission

Always check if a form is submitted using `isset()` or `$_SERVER`.

Example

```
<?php
```

```
if ($_SERVER['REQUEST_METHOD'] == "POST") {
```

```
    echo "Form Submitted Successfully";
```

```
}
```

```
?>
```

6. Form Handling Cycle (Flow)

User fills form → HTML page

Submit button clicked

Data sent to PHP server

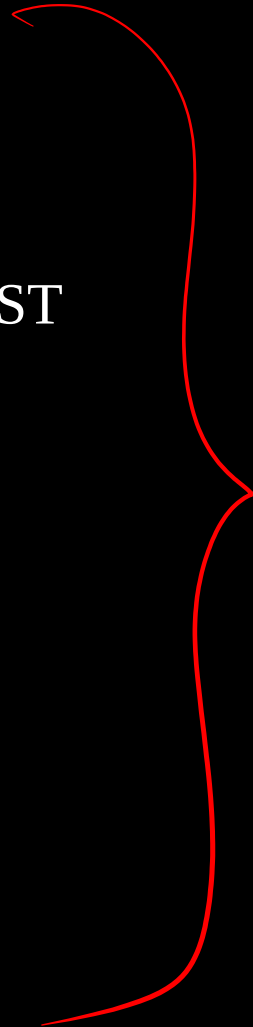
PHP retrieves data using GET/POST

PHP validates & sanitizes input

PHP processes data:

- insert into database
- display output
- send email

Shows success or error message



Home Work

GH_Form.html
GH_Style.css

- 1> Using Html + CSS Create a form for Great Heights 5th Sem Student add mission. (Min 7-8)
- 2> GH_Process.php / all checking & error print successfully store to db

You have to Send

- 1) Registration Page Pic
 - 2) Successful Page Pic
 - 3) Error Showing Pic
- 